

Arithmetic: Our First Interpreter

CS 350

Dr. Joseph Eremondi

February 12, 2025

Interpreters: Overview

Interpreters: Overview

The Church Turing Thesis



Alonzo
Church

*(lambda
calculus)*



Kurt Gödel

*(general
recursive
functions)*



Alan Turing

*(Turing
machines)*

Turing Completeness

Turing Completeness

- The following can all simulate each other:

Turing Completeness

- The following can all simulate each other:
 - Turing Machines

Turing Completeness

- The following can all simulate each other:
 - Turing Machines
 - General-recursive functions

Turing Completeness

- The following can all simulate each other:
 - Turing Machines
 - General-recursive functions
 - Lambda calculus (we'll see later)

Turing Completeness

- The following can all simulate each other:
 - Turing Machines
 - General-recursive functions
 - Lambda calculus (we'll see later)
- We call a programming language that can simulate a Turing machine *Turing Complete*

The Church Turing Thesis

Turing Completeness

- The following can all simulate each other:
 - Turing Machines
 - General-recursive functions
 - Lambda calculus (we'll see later)
- We call a programming language that can simulate a Turing machine *Turing Complete*
 - Any language with `while` loops or recursion is Turing Complete

The Church Turing Thesis

Turing Completeness

- The following can all simulate each other:
 - Turing Machines
 - General-recursive functions
 - Lambda calculus (we'll see later)
- We call a programming language that can simulate a Turing machine *Turing Complete*
 - Any language with `while` loops or recursion is Turing Complete

All Turing Complete Languages can simulate each other

Turing Completeness and Interpreters

- You can write an interpreter for any language in any Turing-complete language

Turing Completeness and Interpreters

- You can write an interpreter for any language in any Turing-complete language
- The features of a language you're interpreting are *completely unrelated* to the features of the language the interpreter is written in

Turing Completeness and Interpreters

- You can write an interpreter for any language in any Turing-complete language
- The features of a language you're interpreting are *completely unrelated* to the features of the language the interpreter is written in
 - Sometimes you can piggyback on the implementation language features, but that's a matter what's *convenient*, not what's *possible*

Keeping it all straight

- The implementation language is NOT the language you're interpreting

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit
- We'll write interpreters for a bunch of small languages

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit
- We'll write interpreters for a bunch of small languages
 - We'll call them "Curly" because we write them with curly brackets

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit
- We'll write interpreters for a bunch of small languages
 - We'll call them "Curly" because we write them with curly brackets
 - Write Curly programs in Racket files using quotation

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit
- We'll write interpreters for a bunch of small languages
 - We'll call them "Curly" because we write them with curly brackets
 - Write Curly programs in Racket files using quotation
 - Backtick turns them into S-Expressions for easy parsing

General Form of an Interpreter

General Form of an Interpreter

```
(define (interp [e : Expr]  
           [x : SomeContext]  
           ...  
           [y : OtherContext])  
  : Value  
  ....)
```

- Expr is the is the AST datatype for whatever language we're interpreting

General Form of an Interpreter

```
(define (interp [e : Expr]
                [x : SomeContext]
                ...
                [y : OtherContext])
  : Value
....)
```

- Expr is the is the AST datatype for whatever language we're interpreting
- What the context arguments and Value datatype are depend on the language

General Form of an Interpreter

```
(define (interp [e : Expr]
                [x : SomeContext]
                ...
                [y : OtherContext])
  : Value
....)
```

- Expr is the AST datatype for whatever language we're interpreting
- What the context arguments and Value datatype are depend on the language
 - Initially we have no context arguments, and Value is very simple

General Form of an Interpreter

```
(define (interp [e : Expr]
               [x : SomeContext]
               ...
               [y : OtherContext])
  : Value
  ....)
```

- Expr is the AST datatype for whatever language we're interpreting
- What the context arguments and Value datatype are depend on the language
 - Initially we have no context arguments, and Value is very simple
 - Will get more complicated as we go through the course

Our First AST Type

- Exp is:

Our First AST Type

- Exp is:
 - Output of parsing

Our First AST Type

- Exp is:
 - Output of parsing
 - Input to interp

Our First AST Type

- Exp is:
 - Output of parsing
 - Input to interp
 - Tree representation of the term to evaluate

Our First AST Type

- Exp is:
 - Output of parsing
 - Input to interp
 - Tree representation of the term to evaluate
- Value is the result of running a program

Our First AST Type

- `Exp` is:
 - Output of parsing
 - Input to `interp`
 - Tree representation of the term to evaluate
- `Value` is the result of running a program
 - Output of `interp`

Our First AST Type

- `Exp` is:
 - Output of parsing
 - Input to `interp`
 - Tree representation of the term to evaluate
- `Value` is the result of running a program
 - Output of `interp`
 - For now, just numbers

Our First AST Type

- `Exp` is:
 - Output of parsing
 - Input to `interp`
 - Tree representation of the term to evaluate
- `Value` is the result of running a program
 - Output of `interp`
 - For now, just numbers

Our First AST Type

- Exp is:
 - Output of parsing
 - Input to interp
 - Tree representation of the term to evaluate
- Value is the result of running a program
 - Output of interp
 - For now, just numbers

```
(define-type-alias Value Number)
(define-type Exp
  (num [n : Number])
  (plus [left : Exp]
        [right : Exp])
  (times [left : Exp]
         [right : Exp]))
```

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!
- `interp` is a recursive function on structure of syntax

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!
- `interp` is a recursive function on structure of syntax
 - Base cases are literals, translate directly into values

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!
- `interp` is a recursive function on structure of syntax
 - Base cases are literals, translate directly into values
 - Recursive cases are operations

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!
- `interp` is a recursive function on structure of syntax
 - Base cases are literals, translate directly into values
 - Recursive cases are operations
 - Interpret sub-expressions recursively

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!
- `interp` is a recursive function on structure of syntax
 - Base cases are literals, translate directly into values
 - Recursive cases are operations
 - Interpret sub-expressions recursively
 - Combine according to value version of the operation

Our First Interpreter

- Interpreting arithmetic, so values are just Flit Number

Our First Interpreter

- Interpreting arithmetic, so values are just Flit Number
 - See Curly-Arith on URCourses

Our First Interpreter

- Interpreting arithmetic, so values are just Flit Number
 - See Curly-Arith on URCourses

Our First Interpreter

- Interpreting arithmetic, so values are just Flit Number
 - See Curly-Arith on URCourses

```
(define (interp [e : Exp] )
  : Value
  (type-case Exp e
    [(num n) n]
    [(plus l r)
     (+ (interp l) (interp r))]
    [(times l r)
     (* (interp l) (interp r))])
(define (run s-exp) (interp (parse s-exp)))
(run `3)
(run `{+ 2 5})
(run `{+ {+ 1 {+ 2 1}} {+ {+ 3 4} {+ 0 1000000}}}))
```

Our First Interpreter

- Interpreting arithmetic, so values are just Flit Number
 - See Curly-Arith on URCourses

```
(define (interp [e : Exp] )
  : Value
  (type-case Exp e
    [(num n) n]
    [(plus l r)
     (+ (interp l) (interp r))]
    [(times l r)
     (* (interp l) (interp r))])
(define (run s-exp) (interp (parse s-exp)))
(run `3)
(run `{+ 2 5})
(run `{+ {+ 1 {+ 2 1}} {+ {+ 3 4} {+ 0 1000000}}}))
```

```
3
7
15
```


Adding features

- Need to update

Adding features

- Need to update
 - AST definition

Adding features

- Need to update
 - AST definition
 - Parser

Adding features

- Need to update
 - AST definition
 - Parser
 - Interpreter

Adding features

- Need to update
 - AST definition
 - Parser
 - Interpreter
- Example: `{if0 cond x y}`

Adding features

- Need to update
 - AST definition
 - Parser
 - Interpreter
- Example: `{if  $\theta$  cond x y}`
 - Evaluates to x if cond evaluates to θ

Adding features

- Need to update
 - AST definition
 - Parser
 - Interpreter
- Example: `{if  $\emptyset$  cond x y}`
 - Evaluates to x if cond evaluates to $\emptyset$
 - Evaluates to y if cond evaluates to anything else

Updating the datatype

Updating the datatype

```
(define-type Expr
  (NumLit [n : Number])
  (Plus [left : Exp]
        [right : Exp])
  (Times [left : Exp]
         [right : Exp]))
;;NEW
(If0 [test : Exp]
     [thenBranch : Exp]
     [elseBranch : Exp])
```

Updating the parser

Updating the parser

```
[(s-exp-match? ~{ifo ANY ANY ANY} s)
 (If0 (parse (second (s-exp->list s)))
      (parse (third (s-exp->list s))
            (parse (fourth (s-exp->list s))))]
```

Updating the interpreter

Updating the interpreter

```
(define (interp [e : Exp] ) : Number
  (type-case Expr e
    [(NumLit n) n]
    [(Plus l r)
     (+ (interp l) (interp r))]
    [(Times l r)
     (* (interp l) (interp r))]

    [(If0 test thenBranch elseBranch)
     (if (= 0 (interp test))
          (interp thenBranch)
          (interp elseBranch)
     )])
```